



Bored Games

Advanced Database Project

Professeur : Nathalie XIAODAN CHIN

Students: Camille DOMMERGUE, Nissanth NADARASSIN,
Harish Arvind SELVAKUMAR

Date : 19/05/2025

Table of content :

Table of content :	1
Our Project Concept :	2
Project Description :	2
Target audience:	2
Functionalities:	2
1. User / visitor :	2
2. Logged in User :	2
3. Admin :	2
Use Case :	3
Mockup :	3
Admin page:	3
User / visitor page:	4
Logged in user page:	4
DataBase Structure :	5
Logical Data Model (LDM):	5
Relationships:	5
Actions :	5
User can:	5
Game can:	6
Event can:	6
Normalisation:	6
1NF (First Normal Form) :	6
2NF (Second Normal Form) :	6
3NF (Third Normal Form) :	7
Conceptual data model (CDM):	8
Advanced Features :	9
Views :	9
Index :	9
Trigger :	9
Function & Procedure :	10
The Website structure :	10
Backend :	10
Structure:	10
Main Technologies :	11
Key Backend Files :	11
Connecting to MySQL:	12
Frontend :	12
Key Pages:	12
Features Implemented :	12
The Challenges faced during this project :	13

Our Project Concept :

Project Description :

This project, "Bored Game", is a website catalog for various board games. The board game library web application serves as a centralized platform for board game enthusiasts to explore, track, and participate in board gaming events. It provides a catalog of board games, helps users manage their personal game collection, rate games and facilitates community engagement through event participation.

Target audience:

A community of enthusiast board game collectors / players.

Functionalities:

There is 3 types of users :

1. User / visitor :

- They can surf through the website and search games based on title or genre
- They can also view the different upcoming events
- They can login

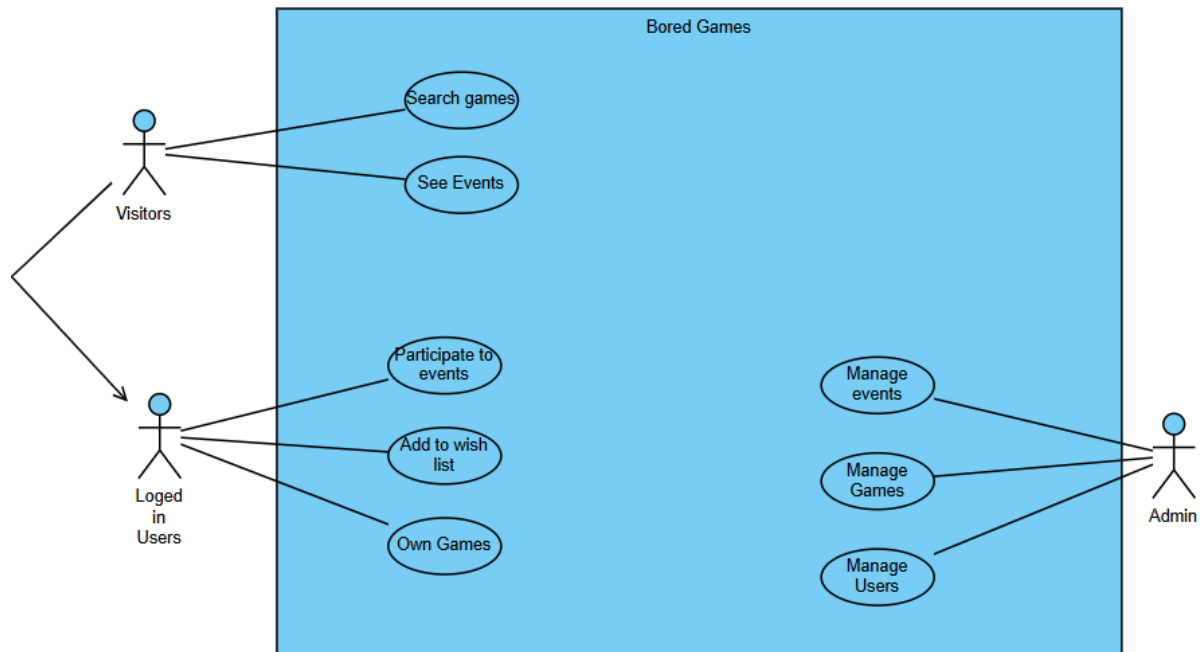
2. Logged in User :

- They have same features as user / visitor
- They can decide to participate to events
- They can rate games
- They can add games to wishlist
- They can notify which games they already own

3. Admin :

- They can manage events & the database
- They can block and unblock users

Use Case :



Mockup :

Admin page:

- Both manage game and manage event are similar in layout.

Admin
Logout

→ Mange Game
 → Manage Event

Create game

Name	Year	Rank	Average ...
------	------	------	-------------

Modify game

Name	Year	Rank	Average ...
Name	Year	Rank	Average ...
⋮	⋮	⋮	⋮

User / visitor page:

BoredGames

Search
Event
Login

Recently added:

...

Game Title

Details

Rating

Upcomming Events

→ 31st Dec

→ 1st Jan

→ 14th Feb

⋮

Username

Password

Submit

New User

Logged in user page:

- The games owned and wishlist pages are similar in layout.

Username

Username
Logout

Games owned
Ratings
Wishlist

Games owned:

☐ ☐ ☐ ☐ ...

Add 

DataBase Structure :

Logical Data Model (LDM):

```
BOARD_GAMES = (game_id NUMBER, name VARCHAR(40), image VARCHAR(255), description VARCHAR(255), year_published INT, min_players SMALLINT, max_players SMALLINT, min_playtime DECIMAL(2,2), max_playtime DECIMAL(2,2), min_age INT, publisher VARCHAR(50), updated_at DATETIME, average_rating DECIMAL(1,2));
GENRES = (genre_id NUMBER, name VARCHAR(40));
User = (user_id NUMBER, is_admin BOOLEAN, username VARCHAR(40), password VARCHAR(40), is_blocked BOOLEAN);
VENUE = (venue_id NUMBER, address VARCHAR(100), name VARCHAR(40), max_capacity INT);
EVENTS = (event_id NUMBER, name VARCHAR(40), description VARCHAR(255), max_participants INT, nb_participant INT, event_time DATETIME, #venue_id);
IsOfGenre = (#game_id, #genre_id);
WishList = (#game_id, #user_id);
GameOwned = (#game_id, #user_id, since DATE);
Rating = (#game_id, #user_id, Stars INT, comment VARCHAR(200));
ParticipateTo = (#event_id, #user_id);
```

Relationships:

1. ParticipateTo: User (0,N) ↔ (0,N) Events
2. TakePlace: Events (0,1) ↔ (1,N) Venue
3. WishList: User (0,N) ↔ (0,N) Board_Games
4. GameOwned: User (0,N) ↔ (0,N) Board_Games
5. Rating: User (0,1) ↔ (0,N) Board_Games
6. IsOfGenre: Board_Games (0,N) ↔ (0,N) Genres

Actions :

User can:

- Participate in multiple Events
- Own multiple Games
- Wish for multiple Games
- Rate multiple Games

Game can:

- Belong to multiple Genres
- Be owned or wished by multiple Users
- Be rated by multiple Users

Event can:

- Takes place at one Venue
- Can have multiple Participants

Normalisation:

1NF (First Normal Form) :

Eliminate Repeating Groups

Ensure that all attributes have atomic (indivisible) values and that there are no repeating groups or arrays.

Problem: The genres column can have multiple values in a single board game.

So, we have a separate table for the genre.

Now, each table has atomic values (no repeating groups or arrays). Each column contains only one value per record.

2NF (Second Normal Form) :

Eliminate Partial Dependencies

All non-key attributes must depend on the entire primary key, not just part of it.

Let's say we had a table:

GameOwned(user_id, game_id, username, type)

Here, the primary key is (user_id, game_id).

But username depends only on user_id, not on both user_id and game_id.

We split the data into two tables:

User(user_id, username, ...)

GameOwned(user_id, game_id, type)

Now, every non-key attribute in GameOwned depends only on the full primary key. All attributes are fully functionally dependent on the entire primary key. In associative tables (e.g., WishList, GameOwned, etc.), no partial dependencies exist because they use composite keys or implicit Foreign Key structure.

3NF (Third Normal Form) :

Eliminate Transitive Dependencies

There should be no transitive dependencies – i.e., non-key attributes should not depend on other non-key attributes.

Let's look for any non-key attribute depending on another non-key attribute.

Example 1: Venue(name, address)

If name → address, then address is transitively dependent on venue_id

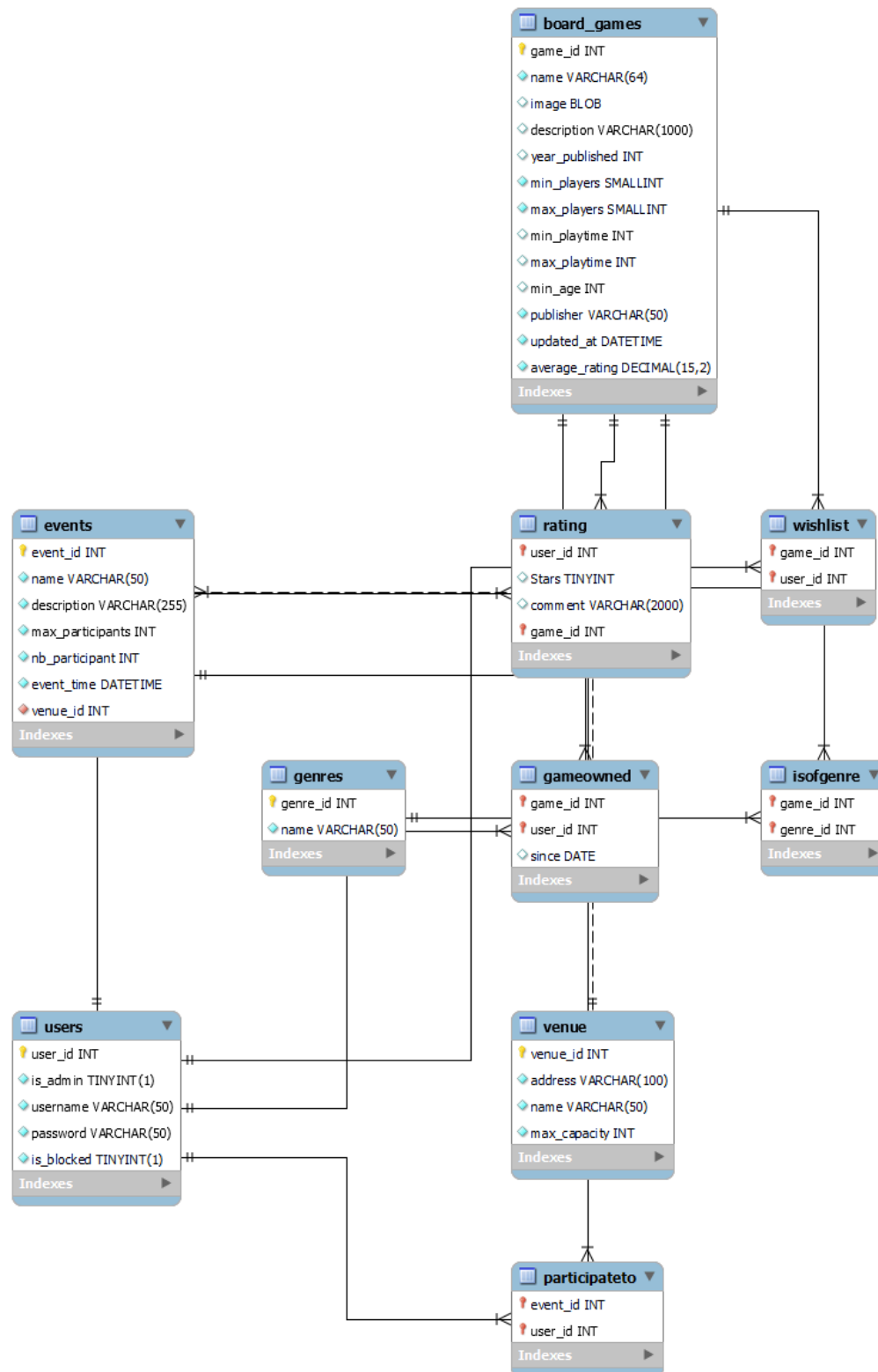
But since venue_id is the PK, and if names are not guaranteed unique, no issue.

Example 2: Board_Games(publisher)

If more info about publishers (like address, website, etc.) were stored, you'd need a separate Publishers table. But right now, publisher is just a string → no transitive dependency.

All attributes are only dependent on the primary key (no transitive dependencies). For example, in Board_Games, average_rating depends only on game_id, not on unrelated non-key fields.

Conceptual data model (CDM):



Advanced Features :

For the advanced Functionalities we think about what will be needed during the website use.

Views :

- Recently added (10 games)
- List of users for admin to view (to Block or unblock)
- List of board games for admin to manage (to Modify and add)
- List of Games owned and the nb of user owning them
- List of Events and nb of user participating
- List of Games and their genre
- List of Genre and nb of games of each

Index :

- User
- Board_Games
- Rating
- Events

We decided to take those index since they will be the most used for queries

Trigger :

- Update BOARD_GAMES.average_rating at new entry in Rating
- Update EVENTS.nb_participant when a new entry is added
- Update BOARD_GAMES.updated_at when board_games modify
- Make sure the EVENTS.max_participant > EVENTS.nb_participant when a new one is added
- Make sure the EVENTS.max_participant <= VENUE.max_capacity

Function & Procedure :

- Search game by year,genre,min_players, max_players, min_playtime, max_playtime,min_age
- List of games in wishlist for an user
- List of games owned for an user
- List of user for events
- Ratings regarding the game
- List of blocked or unblocked users for admin(parameter 0 or 1 if blocked/unblocked)
- Modify events and venues as admin

The Website structure :

Backend :

Structure

The app is built using Flask as the web framework, with MySQL as the relational database. Key components include:

- app.py: The main application file containing all Flask routes and logic.
- setup_and_run.py: A setup script that initializes the database by executing SQL scripts, creating tables, views, triggers, and inserting initial data.
- templates/: Contains Jinja2 HTML templates (e.g., game.html, home.html, login.html) for rendering the frontend.
- static/: Holds static files like CSS and images (if used).

Main Technologies :

Flask: Handles routing, session management, and rendering.

MySQL: Stores data including games, users, ratings, events, and genres.

Flask-MySQLdb: Allows connection and query execution between Flask and MySQL.

Flask-Login: Manages user sessions and authentication.

Flask-WTF & WTForms: Handles form validation securely.

Werkzeug Security: Provides password hashing and checking for secure logins.

Key Backend Files :

app.py

This is the main entry point of the application and contains:

Route definitions:

- Home (/)
- Game Details (/game/<int:game_id>)
- Search Games
- Events
- Login/Register/Logout
- Admin Panel, etc...

User authentication logic

Database queries (e.g., fetching game info, user ratings)

Role-based access (admin/user)

Session and error handling

setup_and_run.py

This script initializes the database by:

Connecting to MySQL

Executing SQL scripts to:

- Create tables and relationships
- Insert initial game and user data
- Define stored procedures, views, and triggers

This ensures the project can be easily set up or reset with one script.

Frontend :

Templates are created using Jinja2 to dynamically render HTML based on backend data.

Key Pages

- Home Page: Displays recently added or popular games.
- Game Detail Page: Shows full game details, genres, image, and community ratings. Logged-in users can rate, comment, and add games to lists.
- Search Page: Allows users to filter games by name, genre, rating, and more.
- Login/Register: Handles user authentication with form validation.
- Admin Dashboard: Lets admins manage games, users, and events.

The UI is clean and intuitive, using basic HTML and CSS for layout, buttons, forms, and lists.

Features Implemented :

Database Views (e.g., Game_Details, Game_Ratings) simplify complex JOIN queries for better performance and readability.

Triggers, Procedures and Functions : Maintain data integrity and automate logic on certain database actions.

Pagination: Game search results are paginated (e.g., 30 per page).

Role Management: Admins and users have different privileges and UI access.

The Challenges faced during this project :

During this project, the most challenging aspect was connecting and integrating the database with the website. Ensuring proper communication between the front-end and back-end, managing secure data access, and implementing functionalities such as data retrieval, user authentication, and protection against SQL injection required careful planning and debugging. Additionally, understanding how to structure queries efficiently and handle potential errors made this part of the project both technically demanding and highly educational.

To address these challenges, we took several key steps:

- **Database Connection:** We used a secure connection method (e.g., using environment variables to store credentials) to link the database to the website's back-end server.
- **Backend Integration:** We implemented server-side scripts (in languages such as PHP, Python, or Node.js) to handle queries between the front-end and the database, ensuring a smooth data flow.
- **SQL Injection Prevention:** We used prepared statements and parameterized queries to protect against SQL injection attacks and ensure secure user input handling.
- **Authentication System:** For features like login and account management, we encrypted passwords using hashing algorithms as bcrypt before storing them in the database.
- **Error Handling & Debugging:** We set up detailed logging and used debugging tools to identify and fix bugs during the integration process.
- **Query Optimization:** We structured our SQL queries efficiently and used indexing where appropriate to enhance performance.

